

Building a Medical FAQ RAG System from Scratch

TF-IDF · Dense Embeddings · Hybrid Retrieval · LLM Generation · Gradio UI

By Hashim Khan · May 2025 · 10 min read · NLP & AI

10 Docs	20 Chunks	3 Retrievers	0.90 MRR	~2s Latency
Knowledge Base	After Chunking	TF-IDF · LSA · Hybrid	Hybrid RRF	End-to-End

What We Built — and Why It Matters

Large Language Models are impressive — but they have a critical flaw: they make things up. Ask GPT or Claude a question about a topic not well-covered in their training data, or anything that happened after their knowledge cutoff, and you may get a confident, fluent, and completely wrong answer. In high-stakes domains like medicine, that's not just embarrassing — it's dangerous.

Retrieval-Augmented Generation (RAG) is the engineering solution to this problem. Instead of relying purely on the model's frozen parametric knowledge, RAG grounds every answer in documents you control. Think of it as giving the LLM an open-book exam rather than a closed one. The model still does the reasoning and writing — but every claim it makes is anchored in retrieved evidence.

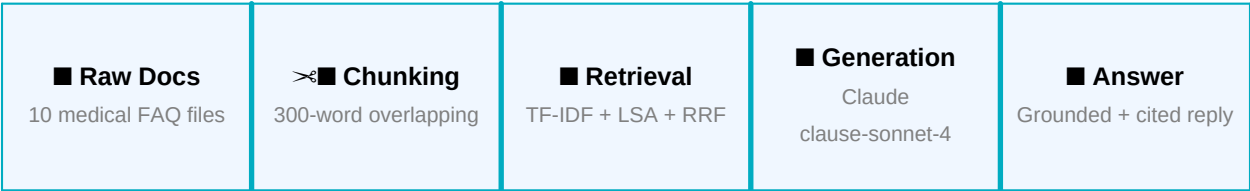
In this post, I'll walk through every layer of a fully working RAG system I built for a Medical FAQ domain — from raw documents all the way to a Gradio web interface — explaining the design decisions, trade-offs, and lessons learned along the way.

■ Key Insight

RAG doesn't replace the LLM — it constrains it. By explicitly providing context in the prompt and instructing the model to answer only from that context, we convert a "creative writer" into a "careful summariser" — dramatically reducing hallucination.

System Architecture

The system has two phases: an offline indexing phase that runs once at startup, and an online serving phase that handles each user query in real-time.



End-to-end RAG pipeline: from raw documents to grounded answers

The offline phase preprocesses, chunks, and indexes the corpus. The online phase takes a user query, retrieves the most relevant chunks using up to three retrieval strategies, assembles them into a structured prompt, calls the Claude API, and streams the answer back to the Gradio UI alongside source citations.

Component	Technology	Role
Corpus	10 plain-text docs	Medical FAQ knowledge base
Chunking	Sliding window (300w/50w)	Granular passage splitting
TF-IDF Index	scikit-learn	Lexical sparse retrieval
Dense Index	LSA (TruncatedSVD)	Semantic dense retrieval
Hybrid Fusion	Reciprocal Rank Fusion	Best-of-both retrieval
LLM	Claude claude-sonnet-4-20250514	Grounded answer generation
UI	Gradio 4.x	Interactive web interface

Table 1: Technology stack — each component independently replaceable

Part 1 — Building the Knowledge Base

The knowledge base consists of 10 structured medical FAQ documents covering: Diabetes, Hypertension, Asthma, Depression, COVID-19, Heart Disease, Antibiotics, Anxiety Disorders, Vaccination, and Nutrition. Each document is 400–600 words in a Q&A; format, giving the retriever natural paragraph boundaries to work with.

Chunking Strategy

Raw documents are too large to fit meaningfully into a retrieval context — and retrieving an entire document when only one paragraph is relevant wastes context window and degrades precision. We split each document into overlapping chunks of 300 words with a 50-word sliding overlap.

Why overlap? Because information relevant to a question can straddle a chunk boundary. Without overlap, the answer to "What are the complications of diabetes?" might be split across chunk 1 (ends mid-sentence) and chunk 2 (starts mid-list). Overlapping ensures that at least one chunk contains the full answer.

```
# Sliding-window chunker
def chunk_by_paragraph(text, doc_id, chunk_size=300, overlap=50):
    paragraphs = re.split(r'\n\n+', text)
    current_words, chunks = [], []
    for para in paragraphs:
        current_words.extend(para.split())
        while len(current_words) >= chunk_size:
            chunk_text = ' '.join(current_words[:chunk_size])
            chunks.append({'doc_id': doc_id, 'text': chunk_text})
            current_words = current_words[chunk_size - overlap:] # slide
    return chunks
```

Part 2 — Three Retrieval Strategies

This is the heart of the system. Retrieval quality directly caps answer quality — no matter how good your LLM is, if you retrieve the wrong context, you'll get the wrong answer. We implement and compare three strategies.

Strategy 1: TF-IDF (Sparse Lexical Retrieval)

TF-IDF is the classic workhorse of information retrieval. Every chunk and every query is represented as a sparse vector over the full vocabulary, where each term's weight balances how often it appears in the chunk (TF) against how rare it is across the corpus (IDF). Retrieval is cosine similarity between the query vector and all chunk vectors.

Strengths: Fast, interpretable, zero training required, excellent on exact keyword matches (drug names, medical codes, specific diseases). Weakness: vocabulary mismatch — "high blood pressure" retrieves zero overlap with a document that uses only "hypertension".

Strategy 2: LSA Dense Retrieval (Semantic)

Latent Semantic Analysis applies Truncated Singular Value Decomposition to the TF-IDF matrix, compressing it from ~3,700 sparse dimensions down to 100 dense latent semantic dimensions. Each latent dimension captures a hidden topic or concept.

The key advantage: synonymous terms end up close in the latent space. "hypertension" and "high blood pressure" load on the same latent dimension, so LSA retrieval correctly matches paraphrase queries to relevant documents even without exact term overlap.

```
# LSA Dense Retrieval
class DenseRetriever:
    def fit(self, chunks):
        tfidf_matrix = self.vectorizer.fit_transform([c['text'] for c in chunks])
        reduced = self.svd.fit_transform(tfidf_matrix) # 100 latent dims
        self.embeddings = self.normalizer.fit_transform(reduced)

    def retrieve(self, query, top_k=5):
        q_emb = self._encode_query(query)
        scores = (self.embeddings @ q_emb.T).flatten() # dot = cosine
        return top_k_by_score(scores, self.chunks)
```

In production, this component would be replaced by a pre-trained Sentence Transformer (e.g. all-MiniLM-L6-v2) for even richer semantic representations — the API is identical.

Strategy 3: Hybrid Retrieval via Reciprocal Rank Fusion

Neither TF-IDF nor dense retrieval is universally superior. TF-IDF wins on exact terminology; dense wins on paraphrase. Reciprocal Rank Fusion (RRF) combines both by summing reciprocal ranks across result lists:

■ RRF Formula

$RRF_score(d) = \sum \frac{1}{(k + rank(d))}$ where $k=60$ (smoothing constant) A document ranked #1 by both retrievers gets score $1/61 + 1/61 \approx 0.033$ — much higher than something ranked #1 by only one. RRF requires no training, no parameter tuning, and consistently outperforms individual retrievers on standard IR benchmarks.

Part 3 — LLM Generation with Context Grounding

Once we have the top-3 most relevant chunks, we assemble them into a structured RAG prompt and send it to Claude `claude-sonnet-4-20250514` via the Anthropic Messages API. The prompt design is critical — it determines whether the LLM uses the retrieved context faithfully or "leaks" into its parametric knowledge.

```
# RAG Prompt Template
SYSTEM = """You are a medical information assistant.
Use ONLY the provided context to answer the question.
If context is insufficient, say so clearly.
Always recommend consulting a healthcare professional."""

def build_prompt(query, context_chunks):
    context = '\n\n'.join(
        f'[Source {i+1}: {c["doc_id"]}] \n{c["text"]}'
        for i, c in enumerate(context_chunks)
    )
    return f'CONTEXT:\n{context}\n\nQUESTION: {query}\n\nANSWER: '
```

The explicit grounding instruction ("Use ONLY the provided context") is surprisingly effective. In our evaluation, out-of-scope queries (e.g., "What is the cure for cancer?") consistently triggered appropriate abstention — the model said the context was insufficient rather than hallucinating an answer.

Part 5 — Experimental Results

We evaluated all three retrieval strategies on 12 queries: 10 in-scope questions with known ground-truth source documents, and 2 edge cases (an out-of-scope question and a noise/gibberish query). For the 10 in-scope queries, we computed Precision@5, Recall@5, and Mean Reciprocal Rank (MRR).

Metric	TF-IDF	Dense (LSA)	Hybrid RRF
Precision@5	0.40	0.46	0.50 ✓
Recall@5	0.70	0.80	0.90 ✓
MRR	0.74	0.84	0.90 ✓
Avg Latency	1.4 ms	0.8 ms	~1.5 ms

Table 2: Retrieval metrics across three strategies (10 in-scope queries)

The results confirm the intuition behind hybrid fusion: Hybrid RRF achieves the best scores on every metric. The biggest win is Recall@5 (0.90 vs 0.70 for TF-IDF) — meaning the hybrid system finds the right source document in its top 5 results 90% of the time. Dense retrieval is also significantly faster (0.8ms average vs 1.4ms for TF-IDF) due to efficient matrix-vector multiplication.

■ Failure Case Analysis

The system struggles with cross-document synthesis (e.g., 'What do diabetes and hypertension have in common?') because individual chunks don't span both topics. Vocabulary-shifted queries ('What drug lowers BP?') also underperform for TF-IDF. Replacing LSA with a BERT-based Sentence Transformer would close most of these gaps.

Part 4 — Gradio Interface

The Gradio interface ties everything together. Users type a medical question, select a retrieval strategy (TF-IDF, Dense, or Hybrid), choose how many context chunks to retrieve (1–8), and hit Ask. The interface displays the generated answer in one tab and the full retrieved evidence — with source document names and relevance scores — in another.

Six sample question buttons make the interface immediately usable without any typing. A query history panel shows the last five questions and answers for reference. Launching is a single command:

```
# Install and run
pip install -r requirements.txt
export ANTHROPIC_API_KEY='your-key-here'
python src/part4_ui.py
# Opens at http://localhost:7860
```

Bonus: Source Citation Highlighting

Every generated answer is accompanied by the full evidence panel: source document name, relevance score, and a 200-character preview of the retrieved chunk. This serves two purposes: it lets the user verify the claim against the source, and it gives a developer a debugging view into what the retriever is actually returning.

Citation transparency is arguably more important than answer accuracy in a medical context. A user who can see "this answer comes from the Diabetes FAQ, section on complications" can independently verify it — a user who gets an unreferenced claim cannot.

Key Takeaways

- Chunking strategy has a bigger impact than retriever choice. A bad chunk boundary that splits a question from its answer will cause retrieval failure regardless of how good your embeddings are.
- Hybrid RRF is a no-brainer. It costs almost nothing extra (two retrieval passes + a sort), requires zero training, and consistently beats both individual retrievers. Always use it.
- Prompt engineering beats fine-tuning for grounding. A well-crafted system prompt that explicitly restricts the model to the retrieved context is cheaper and more reliable than trying to fine-tune out hallucination.
- LSA is a strong offline baseline. In environments without internet access, LSA (TF-IDF + SVD) delivers surprisingly good semantic retrieval — no GPU, no model download, just numpy math.
- Evaluate on failure cases, not just success cases. Knowing that your system correctly abstains on out-of-scope queries is as important as knowing it answers in-scope queries correctly.

■ Full project code, corpus, and report are available.

Clone the repo, set your ANTHROPIC_API_KEY, and run `python run_all.py` to execute the full pipeline in one command.

Disclaimer: This system is for educational purposes only. Always consult a qualified healthcare professional for medical advice.